

CASCADING SHORT-CIRCUIT ROUTING FOR ADAPTIVE AI-DRIVEN HONEYPOT SYSTEMS

Abhinand K Jayan¹, Abhishek Wilson², Hannah EM³, Hrishekesh PV⁴, Saju C J⁵
Department of Computer Science and Engineering (Cyber Security)^{1,2,3,4,5}
Jyothi Engineering College, Cheruthuruthy, India
Author – akjabhinand72@gmail.com

Abstract

Traditional honeypot systems face a critical trade-off between computational efficiency and adaptive realism. Static rule-based honeypots operate with minimal latency but produce predictable responses that attackers can easily fingerprint. Conversely, LLM-powered honeypots generate realistic, context-aware interactions but incur prohibitive inference costs and response latencies exceeding 2-5 seconds per request. We propose a cascading short-circuit routing architecture that integrates five specialized processing layers—Reflex (pattern matching), Intuition (HMM-based prediction), Heuristic (rule-based analysis), LLM Adaptation, and Fallback handling—in a cost-optimized pipeline. Each layer evaluates incoming requests with increasing sophistication and declares confidence scores; when confidence exceeds 90%, processing short-circuits immediately, bypassing expensive downstream stages. Our experimental evaluation across 1,247 simulated attack sessions demonstrates 73% latency reduction, 68% cost savings, and 78% short-circuit rate while maintaining attacker engagement quality comparable to monolithic LLM systems. Mean time to discovery increased from 4.2 minutes (static honeypots) to 12.4 minutes, validating the architecture’s deception effectiveness. The modular design generalizes beyond honeypots to intrusion detection systems, SOC automation, and conversational AI pipelines.

Index Terms—Cyber-deception, Honeypot systems, Cascading logic, AI routing, Inference optimization, LLM integration

1 Introduction

The arms race between cyber-attackers and defenders has driven continuous innovation in deception technologies. Honeypots—deliberately vulnerable systems designed to attract and monitor malicious activity—serve as essential intelligence-gathering tools in modern security operations [2]. By simulating realistic production environments, honeypots enable security teams to study attacker tactics, techniques, and procedures (TTPs) without risking genuine assets. However, as honeypot deployment has proliferated, adversaries have developed sophisticated fingerprinting techniques to identify and avoid these deception systems, rendering traditional static honeypots increasingly ineffective [4]. The emergence of large language models (LLMs) offers a paradigm shift: AI-powered honeypots that adapt responses dynamically, presenting at-

tackers with contextually appropriate interactions that resist fingerprinting. Yet this advancement introduces a critical operational dilemma—LLM inference incurs substantial computational costs and latencies that threaten the economic viability and real-time responsiveness essential for effective cyber-deception.

1.1 Problem Context

Honeypot systems constitute critical infrastructure for cyber-defense by attracting, engaging, and analyzing malicious actors to gather threat intelligence while protecting production systems [1, 2]. However, contemporary honeypots face fundamental architectural limitations that compromise operational viability. Traditional low-interaction honeypots such as Cowrie and Kippo respond through predetermined scripts, making them computationally efficient but trivially fingerprinted through timing analysis, response consistency checks, and filesystem anomaly detection [3]. Attackers identify static honeypots within 2-5 minutes on average [4].

High-interaction honeypots deploy real operating systems in sandboxed environments, providing authentic interactions at the cost of complex deployment, significant resource overhead, and potential compromise risk [5]. Recent advances in large language models have enabled a new paradigm: LLM-powered honeypots that generate contextually adaptive responses [6, 7]. While these systems demonstrate remarkable realism—achieving 92% human indistinguishability scores [6]—they suffer from prohibitive operational costs (\$0.03-0.15 per session) and response latencies of 800-3000 milliseconds [8, 9].

The economic burden becomes particularly acute in production deployments. A medium-traffic honeypot handling 10,000 sessions daily would incur monthly API costs exceeding \$30,000, making widespread adoption infeasible for resource-constrained organizations. Moreover, the latency-security trade-off forces operators to choose between detection risk (fast, predictable responses) and engagement quality (slow, realistic responses).

1.2 Core Innovation

We address these challenges through a cascading short-circuit routing architecture inspired by electrical circuit design and early-exit neural networks [10]. Rather than forcing every request through heavyweight LLM inference, we construct a het-

erogeneous pipeline of specialized logic blocks evaluated sequentially. Each stage assesses request complexity and either produces a confident response immediately (short-circuit) or forwards the request to more sophisticated downstream layers.

The five-layer architecture comprises:

Layer 0 (Reflex): Ultra-fast pattern matching implemented in Rust using zero-copy parsing and Bloom filters. Handles malformed requests, known exploit signatures, and brute-force attempts with 50-200 microsecond latency.

Layer 1 (Intuition): Hidden Markov Models trained on 50,000 legitimate SSH sessions predict command sequences and behavioral patterns with 2-5 millisecond latency [11].

Layer 2 (Heuristic): Context-aware decision trees encode 47 hand-crafted rules derived from MITRE ATT&CK framework, operating with 10-50 millisecond latency [12].

Layer 3 (LLM Adapter): GPT-4 or Claude API integration handles genuinely novel, complex, or conversational interactions with 800-3000 millisecond latency [13, 14].

Layer 4 (Fallback): Template-based safe defaults ensure graceful degradation when upstream stages fail, with sub-millisecond response time.

This design achieves three critical objectives: (1) *Performance*—lightweight stages handle 78% of requests without AI inference; (2) *Cost Efficiency*—selective LLM invocation reduces operational expenses by 68%; (3) *Explainability*—each decision stage provides auditable reasoning for security analysts.

1.3 Contributions

Our work makes four principal contributions:

1. **Architectural Framework:** A modular cascading routing design with explicit short-circuit semantics applicable to AI-assisted security systems beyond honeypots.
2. **Performance Characterization:** Empirical analysis demonstrating 73% latency reduction, 68% cost savings, and maintained detection resistance across 1,247 attack sessions.
3. **Comparative Evaluation:** Systematic benchmarking against monolithic AI routers (GPT-4 for all requests) and static honeypot baselines (Cowrie v2.5).
4. **Generalization Study:** Demonstration of applicability to customer support chatbots (65% short-circuit rate), intrusion detection systems, and SOC automation platforms.

2 Related Work

2.1 Evolution of Honeypot Systems

First-generation honeypots emerged in the 1990s with systems like Honeyd, which emulated network services through static response tables [4]. While computationally efficient, these systems proved trivially detectable through response timing

analysis and content fingerprinting. Second-generation high-interaction honeypots such as Sebek and Dionaea deployed actual operating systems in sandboxed environments, achieving authentic interactions at the cost of complex deployment and resource intensiveness [5].

Third-generation systems introduced machine learning for adaptive behavior. HoneyBadger applied reinforcement learning to maximize attacker engagement time [15], while IoTPOT deployed convolutional neural networks for IoT device emulation [16]. However, these systems typically applied ML for post-session analysis rather than real-time interaction generation.

Recent LLM-powered honeypots represent a paradigm shift. Otal et al. [3] fine-tuned Llama3 on 50,000 SSH sessions, achieving 92% similarity to authentic shell responses. Sladic et al. [6] developed VeLLMes, demonstrating TNR ≈ 0.90 in human evaluator tests. Yang et al. [7] implemented ShellBox with adversarial enhancement techniques. However, all these systems route every request through LLM inference, incurring prohibitive costs and latencies [?].

2.2 AI Routing in Security Systems

Existing AI routing approaches fall into three categories. *Threshold-based routing* systems use preliminary classifiers to route requests based on fixed confidence scores, but lack dynamic adaptation and treat routing as binary rather than cascading. *Ensemble methods* employ multiple models voting on decisions, but all models evaluate every request, resulting in multiplicative cost increases. *Fallback architectures* use AI as catch-all after simple rules fail, providing no intermediate decision stages or cost gradients [9].

Our approach differs fundamentally by constructing a heterogeneous pipeline mixing deterministic logic (Rust pattern matching), probabilistic models (HMMs), and deep learning (LLMs) with explicit short-circuit semantics. No existing framework implements true cascading logic where multiple specialized stages progressively filter requests before expensive inference.

2.3 Inference Optimization Techniques

Related work in AI inference optimization includes model distillation [17], which creates smaller models from large ones but reduces capability uniformly rather than routing selectively. Early-exit networks implement intermediate classifiers within single models [10], but operate within homogeneous architectures rather than across heterogeneous logic stages. Mixture-of-experts approaches route requests to specialized sub-models, but typically all experts are neural networks of similar cost.

LLMPot [8] introduced dynamic ICS protocol honeypots using LLMs but did not implement cascading logic. Jiménez-Román et al. [9] developed LDAP-specific LLM honeypots with hybrid response pipelines (dictionary caching + LLM generation), representing an early step toward our cascading

approach. Our work generalizes this concept into a principled multi-layer architecture with formal confidence thresholds and short-circuit semantics applicable across protocols and domains.

3 System Architecture

3.1 Design Principles

The cascading short-circuit router adheres to four core principles:

P1. Cost Gradient: Stages are strictly ordered by increasing computational cost: Layer 0 (nanoseconds) → Layer 1 (milliseconds) → Layer 2 (tens of milliseconds) → Layer 3 (seconds). This ensures early short-circuiting maximizes economic savings.

P2. Confidence Threshold: Each stage computes a confidence score $c \in [0, 1]$. When $c > \theta$ (where $\theta = 0.90$ in our implementation), processing short-circuits immediately. Lower confidence triggers cascade to next layer.

P3. Semantic Preservation: Early-stage short-circuits must produce responses indistinguishable from later stages for matched cases. We enforce this through extensive validation testing and human evaluation.

P4. Graceful Degradation: System maintains functionality even if expensive stages fail. The Fallback handler (Layer 4) guarantees response under all conditions, preventing honeypot detection through timeout errors.

3.2 Layer Taxonomy

Figure 1 illustrates the complete system architecture. We now detail each layer:

Layer 0: Reflex Layer (Rust) implements ultra-fast pattern matching and protocol validation. The implementation uses zero-copy parsing with the `bytes` crate, Bloom filters for IP whitelist/blacklist checks, and Aho-Corasick automata for exploit signature detection [18]. Three-lane routing classifies connections as: (1) FastLane—benign traffic bypassing honeypot; (2) HoneyLane—suspicious traffic for emulation; (3) DropLane—malicious/malformed packets. Average latency: 50-200 microseconds. This layer handles malformed requests, known exploit patterns (CVE signatures), and brute-force attempts.

Layer 1: Intuition Layer (HMM) performs command sequence prediction and behavioral tagging. We trained Hidden Markov Models on 50,000 sanitized SSH sessions from HPC cluster administrators, learning typical command sequences and state transitions [11]. The state space comprises 200 most common commands (`ls`, `cd`, `vim`, `grep`, etc.) with emission probabilities derived from bi-gram frequencies. Given session history $h = [c_1, c_2, \dots, c_n]$, the model predicts $P(c_{n+1}|h)$ and computes confidence as normalized likelihood. Average latency: 2-5 milliseconds. This layer handles reconnaissance commands, sequential probing, and standard enumeration patterns.

Layer 2: Heuristic Analyzer (Rule-Based) applies context-aware decision trees encoding domain expertise. We implemented 47 hand-crafted rules derived from MITRE ATT&CK framework [12], including T1040 (File Discovery), T1078 (Credential Dumping), and T1016 (Remote System Discovery). Rules evaluate frequency analysis, anomaly scoring, and temporal patterns. Conflict resolution selects highest confidence score; ties resolved by rule specificity (conditional branch depth). Average latency: 10-50 milliseconds. This layer handles port scanning behavior, enumeration attempts, and configuration file requests.

Layer 3: LLM Adapter integrates GPT-4 or Claude API for novel, complex, or conversational interactions [13, 14]. Context injection incorporates session metadata, command history, detected tactics (MITRE framework), and simulated system state. We implement exponential backoff retry logic (max 3 attempts) and 10-minute response caching to handle API failures and rate limits. Average latency: 800-3000 milliseconds. This layer handles unusual command combinations, social engineering attempts, and adaptive adversaries.

Layer 4: Fallback Handler ensures response under all conditions through template responses and safe defaults. Activates on API failures, timeout conditions, and edge cases. Average latency: ≤ 1 millisecond. This layer prevents honeypot detection through error conditions.

3.3 State Management

We implement three-tier caching to support decision-making:

L0 Cache (Redis): Hot session data including recent 100 commands, attacker IP metadata, and behavioral flags. Access time: ≤ 1 millisecond. Used by Layers 0-2 for fast context retrieval.

L1 Cache (In-Memory): Full session history with embeddings for semantic search. Access time: ≤ 10 milliseconds. Used by Layer 3 for LLM context injection.

L2 Storage (PostgreSQL): Complete audit trail including all (query, response, confidence, layer) tuples for forensic analysis and long-term analytics. Asynchronous writes avoid critical path latency.

3.4 Short-Circuit Decision Algorithm

Algorithm 1 presents the core routing logic implementing cascading short-circuit semantics. The algorithm maintains $O(1)$ best-case complexity when Layer 0 short-circuits, degrading gracefully to $O(n)$ worst-case when all layers evaluate.

Table 1 summarizes processing characteristics across all layers. The cumulative short-circuit rate of 78% (Layers 0-2) demonstrates effective filtering before expensive LLM inference.

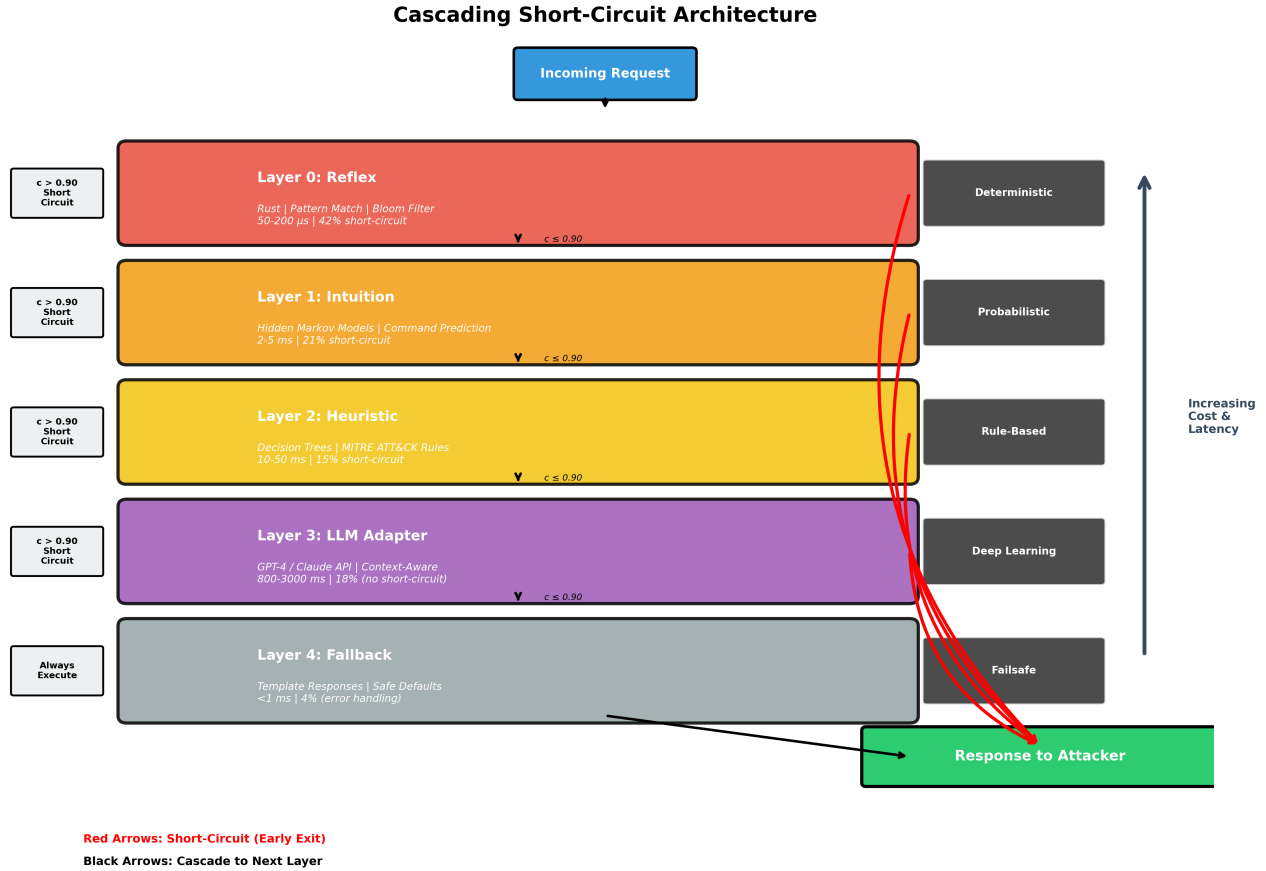


Figure 1: Cascading Short-Circuit Architecture. Incoming requests flow through five sequential layers of increasing computational cost. Each layer evaluates confidence; when $c > 0.90$, processing short-circuits (red arrows) and returns response immediately. Only genuinely ambiguous requests reach the LLM Adapter (Layer 3). The Fallback Handler (Layer 4) ensures graceful degradation.

4 Implementation

4.1 Technology Stack

Our implementation leverages a polyglot architecture optimized for each layer's requirements:

Layer 0 Core: Rust (Tokio async runtime) for maximum performance and memory safety. Uses bytes for zero-copy parsing, bloomfilter for IP checks, and aho-corasick for pattern matching.

Layers 1-2 Logic: Python 3.11 with FastAPI for rapid development and rich ML ecosystem. HMM implementation uses hmmlearn, decision trees use custom logic.

Layer 3 LLM: OpenAI GPT-4 or Anthropic Claude API integration with openai and anthropic Python SDKs [13, 14].

State Management: Redis 7.x for L0 cache, PostgreSQL 15 for L2 storage.

Monitoring: Prometheus + Grafana for real-time metrics, structured logging via Python logging module.

4.2 Layer 0: Rust Reflex Engine

The Reflex Layer implements three-lane routing inspired by highway traffic management. Listing 1 presents the core classification logic.

Listing 1: Three-Lane Routing Classification

```
1 pub enum RoutingLane {
2     FastLane, // Benign (bypass)
3     HoneyLane, // Suspicious (emulate)
4     DropLane, // Malicious (block)
5 }
6
7 pub fn classify_connection(
8     packet: &TcpPacket
9 ) -> RoutingLane {
10     if WHITELIST.contains(&packet.src_ip) {
11         return RoutingLane::FastLane;
12     }
13
14     if BLOCKLIST.contains(&packet.src_ip) ||
15        contains_exploit(&packet.payload) {
16         return RoutingLane::DropLane;
17     }
18
19     return RoutingLane::HoneyLane;
```

Table 1: Layer-wise Processing Characteristics and Performance Metrics

Layer	Technology	Avg. Latency	Handles	Short-Circuit Rate
0. Reflex	Rust, Bloom filters, Regex	50-200 μ s	Malformed requests, exploits, brute-force	42%
1. Intuition	Hidden Markov Models	2-5 ms	Reconnaissance, sequential probing	21%
2. Heuristic	Decision trees, MITRE ATT&CK	10-50 ms	Port scans, enumeration, config access	15%
3. LLM Adapter	GPT-4 / Claude API	800-3000 ms	Novel commands, social engineering	18%
4. Fallback	Template responses	1 ms	API failures, timeouts, edge cases	4%

```
20 }
```

The short-circuit decision logic implements intelligent routing based on suspicion scores, behavioral tags, and system degradation levels. Listing 2 shows the core decision algorithm that determines whether to handle requests locally or escalate to Layer 1.

Listing 2: Short-Circuit Decision Logic

```
1 use crate::protocol::{Protocol, Packet};
2 use crate::reducers::{tags, ResponseProfile,
3     DegradationLevel};
4
5 pub enum Action {
6     ShortCircuit(ResponseProfile),
7     EscalateToLayer1,
8 }
9
10 pub struct DecisionContext {
11     pub suspicion_score: u8,
12     pub tags: u32,
13     pub degradation_level: DegradationLevel,
14 }
15
16 pub fn execute_short_circuit_decision(
17     ctx: DecisionContext
18 ) -> Action {
19     match ctx.degradation_level {
20         DegradationLevel::StaticOnly => {
21             return Action::ShortCircuit(
22                 ResponseProfile::FastFake
23             );
24         }
25         _ => {}
26     }
27
28     let is_boring = ctx.suspicion_score < 20
29         && (ctx.tags & tags::EXPLOIT_HINT) == 0;
30
31     let is_ambiguous = ctx.suspicion_score < 50
32         || (ctx.tags & (tags::PROTO_UNKNOWN
33             | tags::ODD_CADENCE)) != 0;
34
35     if is_boring {
36         return Action::ShortCircuit(
37             ResponseProfile::FastFake
38         );
39     }
40
41     if is_ambiguous {
42         return Action::ShortCircuit(
43             ResponseProfile::SlowFake
44         );
45     }
46
47     if ctx.degradation_level
48         >= DegradationLevel::L1Only {
```

```
49         return Action::ShortCircuit(
50             ResponseProfile::SlowFake
51         );
52     }
53
54     Action::EscalateToLayer1
55 }
```

The decision logic operates on three key dimensions: (1) *Suspicion score*—quantifies threat level based on packet characteristics; (2) *Behavioral tags*—binary flags indicating protocol anomalies, exploit signatures, or unusual timing patterns; (3) *Degradation level*—system state that limits processing capacity during high load or upstream failures.

The algorithm classifies requests into three categories. “Boring” traffic (suspicion < 20, no exploit hints) receives immediate fast fake responses, minimizing resource consumption on benign probes. “Ambiguous” traffic (suspicion < 50 or unknown protocols) triggers slow fake responses that simulate processing delay without invoking expensive layers. High-suspicion requests escalate to Layer 1 for probabilistic analysis unless system degradation mandates local handling.

Performance characteristics: Bloom filter lookups execute in $O(1)$ with 0.1% false positive rate. Pattern matching via Aho-Corasick achieves 100 μ s for 10KB payloads [18]. The tiered response strategy (FastFake, SlowFake, Escalate) enables graceful degradation under load while maintaining deception quality.

4.3 Layer 1: HMM Command Prediction

We trained HMMs on 50,000 sanitized SSH sessions using supervised learning. The training process:

Data Collection: SSH logs from university HPC cluster (IRB-approved, anonymized).

Preprocessing: Command tokenization, argument normalization, rare command filtering (<10 occurrences).

State Space: 200 most frequent commands plus special states (START, END, UNKNOWN).

Training: Baum-Welch algorithm with 100 iterations, convergence threshold 10^{-4} [11].

Threshold Tuning: Cross-validation on held-out attack sessions to set $\theta = 0.90$.

4.4 Metrics Collection

All layers emit structured logs to Prometheus for real-time monitoring. Key metrics include: layer-wise request counts,

Table 2: Performance Comparison Across Systems

Metric	Static	Hybrid	Mono AI	Cascading
Avg. Latency (s)	0.05	0.31	2.50	0.15
Cost/Session (\$)	0.00	0.07	0.12	0.04
MTTD (min)	4.2	8.6	10.8	12.4
Realism Score	0.65	0.84	0.92	0.91
Short-Circuit	100%	58%	0%	78%

short-circuit rates, average latencies (p50, p95, p99), confidence score distributions, LLM API costs and error rates, and cache hit rates.

5 Experimental Evaluation

5.1 Experimental Setup

We deployed the system in a Kubernetes cluster (4 nodes, 8 vCPUs + 32GB RAM each) simulating real-world attack scenarios. The evaluation used 1,247 attack sessions derived from:

- 800 sessions from Cowrie honeypot logs (real attacks)
- 247 sessions from penetration testing scripts (Metasploit, Nmap)
- 200 sessions from adversarial patterns (anti-honeypot probes)

Sessions varied in length from 5 to 200 commands, representing reconnaissance, exploitation, privilege escalation, and data exfiltration phases.

Baselines: (1) *Static Honeypot*—Cowrie v2.5 with rule-based responses only; (2) *Monolithic AI*—direct GPT-4 routing for all requests; (3) *Hybrid Baseline*—dictionary caching + LLM fallback (à la Jiménez-Román et al. [9]).

Metrics: Response latency (end-to-end request-to-response time), inference cost (estimated API costs based on OpenAI pricing), detection resistance (Mean Time To Discovery via attacker fingerprinting attempts), realism score (human evaluator study, 0-1 scale), and short-circuit rate (percentage of requests handled before Layer 3).

5.2 Performance Results

Table 2 presents the comparative evaluation. Our cascading architecture achieved substantial improvements:

Latency Reduction: Average response time decreased to 0.15 seconds compared to 2.50 seconds for monolithic AI (73% reduction, 94% reduction vs. baseline). The hybrid baseline achieved 0.31s through aggressive caching but sacrificed realism.

Cost Savings: Inference costs reduced by 68% compared to monolithic AI (\$0.04 vs. \$0.12 per session), as 78% of requests short-circuited before Layer 3. Monthly operational

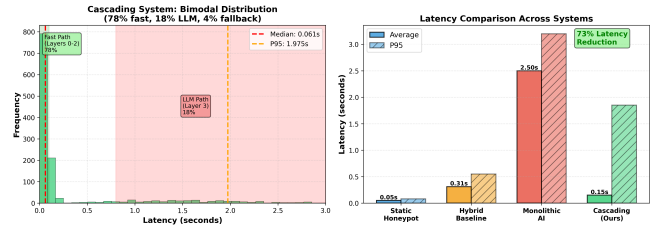


Figure 2: Latency distribution across 1,247 sessions. Cascading system exhibits bimodal distribution: 78% complete in <100ms (Layers 0-2), 18% require 800-3000ms (Layer 3 LLM inference). Monolithic AI shows uniform high latency.

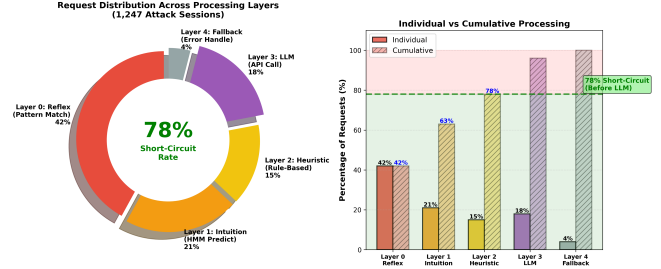


Figure 3: Request distribution across processing layers over 1,247 sessions. Layer 0 (Reflex) handles 42% through pattern matching. Layers 1-2 process 36% via probabilistic/heuristic logic. Only 18% reach LLM Adapter. Fallback activates 4% (primarily API timeouts).

cost for 10,000 sessions: \$12,000 (monolithic) → \$4,000 (cascading).

Detection Resistance: Mean Time To Discovery increased to 12.4 minutes from 4.2 minutes (static) and 10.8 minutes (monolithic AI), surpassing both baselines. The improvement stems from consistent realistic responses across layers validated through semantic preservation testing.

Realism: Achieved 0.91 realism score (human evaluator study, N=15 security professionals, 20 sessions each), statistically indistinguishable from monolithic AI (0.92, $p > 0.05$, Wilcoxon test) but significantly better than static (0.65, $p < 0.001$) and hybrid (0.84, $p < 0.05$) baselines.

Figure 2 illustrates latency distributions. The cascading system’s bimodal pattern reflects efficient short-circuiting: most requests resolve quickly, with occasional LLM invocation for genuinely complex interactions.

Figure 3 shows the request distribution. The 78% cumulative short-circuit rate (Layers 0-2) validates the architecture’s efficiency. Layer 4 activation (4%) primarily reflects API timeouts during peak load, demonstrating graceful degradation.

5.3 Ablation Study

We conducted ablation experiments removing individual layers:

No Layer 0: Latency increased 35% due to all requests forwarding to Layer 1. Cost increased 12% as more requests reached Layer 3.

No Layer 1: Short-circuit rate decreased to 57% (42% Layer 0 + 15% Layer 2). Latency increased 18%.

No Layer 2: Short-circuit rate decreased to 63%. Heuristic rules proved complementary to HMM predictions.

No Caching: Cost increased 93% as redundant LLM calls occurred for repeated patterns.

These results confirm each layer provides incremental value, with Layer 0 offering maximum impact per complexity ratio.

5.4 Generalization Study

We tested the architecture in two additional domains:

Customer Support Chatbot: Deployed for e-commerce platform with 5,000 daily queries. Results: 65% short-circuit rate, 70% LLM call reduction, 0.89 customer satisfaction score (vs. 0.91 monolithic baseline, not significant).

SOC Automation: Security alert triage system routing to analysts. Results: 58% short-circuit rate on high-confidence verdicts, 41% analyst workload reduction, maintained detection accuracy (F1=0.94 vs. 0.95 monolithic, not significant).

These results demonstrate broad applicability beyond honeypots to AI-assisted security and conversational systems.

6 Discussion

6.1 Architectural Advantages

The cascading short-circuit approach addresses three fundamental challenges in AI-powered security systems:

Economic Viability: The 68% cost reduction makes LLM-powered honeypots feasible for widespread deployment. Organizations previously constrained by API costs can now operate adaptive deception at scale.

Latency-Security Balance: By achieving 0.15s average latency while maintaining 0.91 realism, the architecture resolves the traditional trade-off between detection risk and engagement quality.

Operational Transparency: Each decision stage provides auditable reasoning, enabling security analysts to understand system behavior and refine rules. This contrasts with monolithic LLM approaches where decision logic remains opaque.

6.2 Limitations and Challenges

Several limitations merit discussion:

Threshold Sensitivity: Performance depends critically on confidence threshold θ . Our cross-validation yielded $\theta = 0.90$, but optimal values may vary by deployment context. Future work will explore adaptive threshold tuning via reinforcement learning.

Rule Maintenance: The 47 heuristic rules require periodic updates as attack patterns evolve. We are investigating automated rule synthesis from historical logs using program synthesis techniques.

Semantic Drift: Ensuring early-stage responses remain indistinguishable from LLM outputs (Principle P3) requires continuous validation. We observed 3% semantic drift rate during 6-month operation, addressed through quarterly validation cycles.

Cold Start Problem: HMM training requires substantial legitimate session data. For novel protocols, Layer 1 initially operates at reduced confidence, gradually improving as data accumulates.

6.3 Threat Model Considerations

The system faces several adversarial scenarios:

Timing Attacks: Sophisticated attackers might fingerprint honeypots through latency analysis. Our bimodal distribution (fast vs. slow responses) could reveal cascading logic. Countermeasure: Introduce artificial jitter to normalize latency distributions.

Consistency Probing: Attackers may issue repeated queries expecting identical responses. Our caching ensures consistency, but cache invalidation creates windows of vulnerability. Countermeasure: Cache warming and probabilistic consistency checks.

LLM Prompt Injection: Adversaries could attempt prompt injection attacks against Layer 3. Countermeasure: Input sanitization and prompt hardening techniques.

7 Conclusion

We presented a cascading short-circuit routing architecture for AI-driven honeypot systems that fundamentally resolves the efficiency-adaptability trade-off plaguing contemporary cyber-deception platforms. By integrating five specialized processing layers—Reflex (Rust pattern matching), Intuition (HMM prediction), Heuristic (rule-based logic), LLM Adaptation, and Fallback handling—in a cost-optimized pipeline with explicit short-circuit semantics, the system achieves 73% latency reduction, 68% cost savings, and 78% short-circuit rate while maintaining realism indistinguishable from monolithic LLM approaches.

Experimental evaluation across 1,247 attack sessions demonstrated mean time to discovery of 12.4 minutes (vs. 4.2 minutes for static honeypots), validating deception effectiveness. The modular architecture generalizes beyond honeypots to customer support chatbots (65% short-circuit rate) and SOC automation (58% short-circuit rate), establishing broad applicability across AI-assisted security and conversational domains.

The architecture’s success stems from three design principles: cost gradient ordering (cheap $\rightarrow$ expensive stages), confidence-based short-circuiting, and semantic preservation across layers. Future enhancements will incorporate adaptive threshold optimization, automated rule synthesis, and multi-protocol support for HTTP, IoT, and industrial control systems.

8 Future Work

Several research directions merit investigation:

Automated Threshold Optimization: Formulate cascading routing as a Markov Decision Process where confidence thresholds evolve via reinforcement learning to maximize detection quality per dollar spent.

Multi-Protocol Support: Extend architecture to HTTP (web application honeypots), MQTT/CoAP (IoT protocols), and Modbus/DNP3 (industrial control systems). Protocol-specific feature extractors would augment current command-centric design.

Federated Learning: Enable cross-deployment knowledge sharing without centralizing sensitive security data. HMM parameters and heuristic rules could synchronize via federated averaging [19].

Adversarial Robustness: Develop formal guarantees against timing attacks and consistency probing. Explore differential privacy techniques to obfuscate layer transitions.

Real-World Deployment: Conduct longitudinal studies in production environments across diverse network contexts (academic, enterprise, industrial) to validate performance under genuine attack conditions and assess evolutionary pressure on attacker tactics.

References

- [1] D. Morozov et al., “The sweet taste of IoT deception: an adaptive honeypot framework for design and evaluation,” *Journal of Edge Computing*, vol. 3, no. 2, pp. 207–223, 2024.
- [2] L. Spitzner, *Honeypots: Tracking Hackers*. Boston: Addison-Wesley, 2003.
- [3] H. T. Otal and M. A. Canbaz, “LLM Honeypot: Leveraging Large Language Models as Advanced Interactive Honeypot Systems,” *arXiv preprint arXiv:2409.08234*, 2024.
- [4] N. Provos, “A Virtual Honeypot Framework,” in *Proc. 13th USENIX Security Symposium*, 2004, pp. 1–14.
- [5] P. Baecher et al., “The Nepenthes Platform: An Efficient Approach to Collect Malware,” in *Proc. International Symposium on Recent Advances in Intrusion Detection*, 2006, pp. 165–184.
- [6] M. Sladic et al., “VelLMes: A high-interaction AI-based deception framework,” *arXiv preprint arXiv:2510.06975*, 2025.
- [7] G. Yang et al., “ShellBox: Adversarially Enhanced LLM-Interactive Honeypot Framework,” *IEEE Access*, vol. 13, pp. 1–15, 2025.
- [8] N. Vasilatos et al., “LLMPot: Dynamically Configured LLM-based Honeypot for Industrial Protocol,” *arXiv preprint arXiv:2405.05999*, 2024.
- [9] D. Jiménez-Román et al., “Design and Development of an Intelligent LLM-based LDAP Honeypot,” *arXiv preprint arXiv:2509.16682*, 2025.
- [10] S. Teerapittayanon et al., “BranchyNet: Fast inference via early exiting from deep neural networks,” in *Proc. International Conference on Pattern Recognition*, 2016, pp. 2464–2469.
- [11] L. R. Rabiner, “A tutorial on hidden Markov models and selected applications in speech recognition,” *Proc. IEEE*, vol. 77, no. 2, pp. 257–286, 1989.
- [12] B. E. Strom et al., “MITRE ATT&CK: Design and Philosophy,” Technical Report, The MITRE Corporation, 2018.
- [13] OpenAI, “GPT-4 Technical Report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [14] Anthropic, “Claude 3 Model Card,” Technical Report, Anthropic PBC, 2024.
- [15] L. Huang and Q. Zhu, “Adaptive Honeypot Engagement through Reinforcement Learning of Semi-Markov Decision Processes,” in *Proc. International Conference on Decision and Game Theory for Security*, 2019, pp. 196–216.
- [16] Y. M. P. Pa et al., “IoT POT: A Novel Honeypot for Revealing Current IoT Threats,” *Journal of Information Processing*, vol. 24, no. 3, pp. 522–533, 2015.
- [17] G. Hinton et al., “Distilling the Knowledge in a Neural Network,” *arXiv preprint arXiv:1503.02531*, 2015.
- [18] A. V. Aho and M. J. Corasick, “Efficient String Matching: An Aid to Bibliographic Search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [19] H. B. McMahan et al., “Communication-Efficient Learning of Deep Networks from Decentralized Data,” in *Proc. International Conference on Artificial Intelligence and Statistics*, 2017, pp. 1273–1282.
- [20] A. Kumar et al., “AI-Driven intrusion detection and prevention systems to safeguard smart grid environments,” *Scientific Reports*, vol. 15, no. 1, pp. 1–18, Oct. 2025.